



HABIB UNIVERSITY

Data Structures & Algorithms

CS/CE 102/171 Spring 2025

Instructor: Maria Samad

Quiz # 1 Solution

QUESTION 1: (20 marks)

You are given a queue of numbers called **QueueNum** containing 'n' elements, where 'n' is at least 3. The program calculates the sum of numbers starting from the current number until the end. The result is then stored in a new Queue, called **QueueSum**, also of size 'n' but initially empty. Assume 'n' is correctly given to you, so you may use it directly in your algorithm, without having to do any error checking or taking input. However, do declare it in the algorithm to define what it is, including the queues that you may use for your program.

For example,

- Assume **QueueNum** = [1, 2, 3, 4, 5, 6] then the resulting queue, i.e. **QueueSum** = [21, 20, 18, 15, 11, 6]
- This means that for first sum, it will be calculated as $1 + 2 + 3 + 4 + 5 + 6 = 21$, and then for next entry, it will be $2 + 3 + 4 + 5 + 6 = 20$, and then $3 + 4 + 5 + 6 = 18$, and so on

You must implement the above tasks by writing an algorithm for it.

Restrictions:

- Do not **HARD CODE** your design for the given example. The algorithm should work for any valid sized queues of numbers.
- Assume the number of elements in the Queue is 'n', where $n > 2$, so there will always be at least three numbers in a queue to begin with.
- Assume None represents empty slots in all the data structures used for this question
- You are **NOT** allowed to use **any** additional **ADT**, including ListADT
- **DO NOT USE ANY OTHER DATA STRUCTURES INCLUDING ListADT/Python lists apart from the ones permitted to use**
- **REMEMBER:** Stacks and Queues are non-iterable and non-traversing data structures, so you **CANNOT** and **MUST NOT** do indexing within the data structures to access the elements

SOLUTION:

#Required variable declarations and initialization

- n – an integer representing the number of elements in the input queue, i.e., QueueNum
- QueueNum = [None] * n
- QueueSum = [None] * n

- sum_elements – an integer variable to keep track of the sum, initialized to 0
- lenQueueNum – number of elements in QueueNum initially set to n, as there are n elements in QueueNum

- for n times, do the following:
 - element = dequeue from QueueNum
 - sum_elements = sum_elements + element

 - for lenQueueNum – 1 times, do the following:
 - element = dequeue from QueueNum
 - sum_elements = sum_elements + element
 - enqueue element back in QueueNum

 - Enqueue sum_elements in QueueSum
 - lenQueueNum = updated number of elements in QueueNum
 - sum_element = 0

QUESTION 2: (5 marks)

Assume non-circular ListADT of size 4, with empty slots represented as None values in ADT. Fill the given table for the specified ListADT functions

SOLUTION:

Operation	Return Value	ListADT Contents
L.is_full()	False	[None, None, None, None]
L.insert_last('C')	–	[C, None, None, None]
L.insert_last('A')	–	[C, A, None, None]
length(L)	2	[C, A, None, None]
L.insert_first('B')	–	[B, C, A, None]
L.remove_last()	A	[B, C, None, None]
L.insert_last('S')	–	[B, C, S, None]
L.is_full()	False	[B, C, S, None]
L.remove_first()	B	[C, S, None, None]
length(L)	2	[C, S, None, None]
L.remove_first()	C	[S, None, None, None]
L.remove_last()	S	[None, None, None, None]
L.is_empty()	True	[None, None, None, None]

